

Unit -3

Apply the **useContext** hook to share data across multiple components.

Construct a small code snippet showing **data passed from parent to child via props**.

Differentiate between **props** and **state** in React component communication.

Evaluate the usefulness of **Context API** in a medium-sized application.

State the meaning of **Route Params** in React Router v6.

Create a React example that demonstrates **Route Params** for displaying product details.

Summarize the steps to implement **Nested Routes** in a blog application.

Analyze the advantages of **Custom Hooks** compared to normal hooks.

Illustrate a simple **Custom Hook** that manages form input logic.

List two applications of **LocalStorage** in React development.

Compare **Context API** and **Custom Hooks** as solutions for component communication.

Conclude the importance of **Routing and Component Communication** in building scalable React applications.

Define the concept of **React Router v6** in single-page applications.

Analyze the methods of component communication through **Props, Context API, and Custom Hooks**, and conclude the most suitable approach for large-scale applications.

acer

A S P I R E

List two applications of **LocalStorage** in React development.

Compare **Context API** and **Custom Hooks** as solutions for component communication.

Conclude the importance of **Routing and Component Communication** in building scalable React applications.

Define the concept of **React Router v6** in single-page applications.

Analyze the methods of component communication through **Props, Context API, and Custom Hooks**, and conclude the most suitable approach for large-scale applications.

Evaluate the design of a **product listing feature** in a React application with reference to **Search, Filters, and Pagination**, and justify the approach that enhances user experience.

Mention the purpose of using the **useContext()** hook in React.

State the role of **Route Params** in dynamic routing.

Mention two benefits of using **Nested Routes** in React applications.

Explain the function of the **BrowserRouter** component.

150% Title Arial 26 + B I U A [edit icon] [link icon] [image icon] [list icon] [table icon] [text icon] [undo icon] [redo icon] Editing

tabs +

3

4

Describe the purpose of the **Link** component in React Router.

Illustrate the process of **navigating between components** using React Router v6.

Differentiate between **relative navigation** and **absolute navigation** in React Router.

Summarize the need for **data communication between parent and child components** in React.

Describe the limitations of **prop drilling** in component communication.

Explain the role of **Context API** in avoiding prop drilling.

State one benefit of using **Axios** as compared to the fetch API.

Unit 4

Demonstrate the process of implementing **Pagination** in a React application with product listings.

Explain the difference between **static search** and **dynamic search** in React applications.

Apply the concept of **filters** to refine product data displayed in a React component.

Summarize the process of storing and retrieving values from **LocalStorage**.

Analyze the difference between **LocalStorage** and **SessionStorage** in React usage.

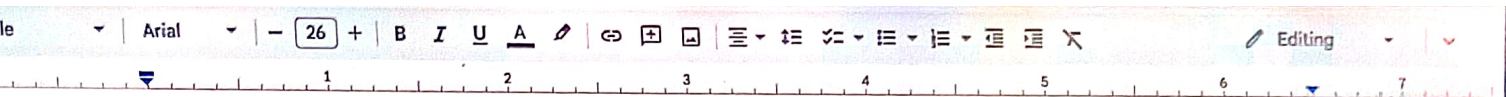
Demonstrate the use of **LocalStorage** for persisting a user's login state.

Explain the concept of **Custom Hooks** and their purpose in React applications.

Describe the process of maintaining **loading states** while integrating APIs in React.

Apply the concept of **LocalStorage** to store and retrieve a user's theme preference (dark / light).

Construct a simple **Custom Hook** that fetches data from an API and returns the response.



Construct a simple **Custom Hook** that fetches data from an API and returns the response.

I

Evaluate the role of **Custom Hooks** in improving code reusability and maintainability.

Apply the concepts of **Search, Filters, and Pagination** to design a scalable product catalog page.

Conclude the importance of **API Integration** and **state management** in building real-world React applications.

Define the role of **fetch()** in retrieving data from APIs in React.

State one advantage of using **Axios** for API integration.

State one advantage of using **Axios** for API integration.

Describe the concept of **asynchronous programming** in the context of React applications.

Explain the function of **Promises** in handling API responses.

Illustrate the use of **async/await syntax** for fetching data in React.

Summarize the need for maintaining **loading states** in API-driven applications.

Describe the steps to implement **API error handling** in React.

Differentiate between **data passing through Props** and **data sharing through Context API**.

Illustrate the concept of **Nested Routes** in React Router v6 with an example.

Differentiate between **client-side error handling** and **server-side error handling**.

Construct a React example that fetches and displays a list of users with a **loading indicator**.

Demonstrate the process of implementing **Pagination** in a React application with product listings.

Create a functional React component to integrate API data using **Axios**, implement **loading and error handling states**, and display the output in a structured list format.